CST2550 GROUP COURSEWORK

Raj - Muzzamil - Ali – Aymen-Thriambakeshwar

(AM3951@live.mdx.ac.uk , AA5715@live.mdx.ac.uk , mh2049@live.mdx.ac.uk , TM1102@live.mdx.ac.uk, Ra1506@live.mdx.ac.uk)

(M00932446, M01046020, M01032133, M00949012, M01025875)

The Video Renting System (VRS)

The VRS (Video Renting System) is a C# .NET application built to give independent creators, small publishers, and local distributors a way to upload and rent out their content. The idea was to build something like a marketplace for video rentals where it's not just the big studios that get to publish, anyone can put their work up.

Under the hood it uses a custom AVL tree for searching, a hash table for fast ID lookups, and a SQLite database to keep data between sessions. The team started out with a WinForms version but quickly realised it didn't fit what they were trying to build, a desktop app that only runs on one machine doesn't really work for a multi-user platform. So they switched to a React and TypeScript frontend backed by an ASP.NET Core Web API, which made a lot more sense. The goal was to end up with something that actually works like a real rental platform while also showing proper use of data structures and algorithms.

This report covers the design choices the team made, why each data structure was picked, how the algorithms work, the testing approach, and what the team would do differently if starting again.

Team Roles & Contributions

Raj Aryan – Team Leader: Ran the sprint planning and kept the team on track throughout the project. Raj built his own VRS console app as a prototype and pushed it to the repo so everyone could see how the data structures and rental logic fit together. His build had a multi-role login system (Fan, Artist, Admin), account creation logic, salted password hashing, AVL tree and hash table integration, full menu workflows, renting, returning, and searching functionality, as well as SQL connectivity testing, ensuring full compliance with coursework rules such as using custom data structures, SQL, and avoiding third-party libraries. Having it on the repo early meant the team had something concrete to look at and agree on before writing the final version.

Muzammil – Secretary: Muzammil kept track of the meeting minutes, sprint logs, and decisions throughout the project and handled the team's scheduling and communication. He also built his own demo program to work through the data structure logic and user account side of things and pushed it to the repo so the team had it as a reference. He put together a UI prototype for the web interface as well which helped the team agree on the design.

Ali – Developer 1: Ali was the main developer on the project. He went through every prototype the team had pushed to the repo Raj's console app, Subbs' database prototype, and Aymen's test cases and took what worked from each one to shape the final design. He started by building a WinForms version to lock in the data structure and database logic, then moved to the full web-based stack that became the final submission. He built the ASP.NET Core Web API and the full React and TypeScript frontend, covering the customer marketplace, the publisher studio, and the admin panel. He also built the AI chatbot that lets users talk to the system in plain English, that was the optional extra marks piece in the brief. All the custom data structures were wired into the live app and connected to the SQLite database, and he wrote the full SQL schema making sure nothing was hardcoded. He also ran the GitHub repo, dealt with merge conflicts, made sure everyone's commits were in, and put together the README, the demo video, and the submission docs.

Thriambakeshwar – Developer 2: Thriambakeshwar worked across both the frontend and backend side of the project. He put together his own prototype that focused on the SQL database side and how data entry and validation should work, then committed it to the repo for the team to look at. That prototype helped spot a few validation issues early on and shaped how the database layer ended up being built. He then jumped in on UI screens, form validation, and SQL work in the shared codebase, helped with debugging, and generally kept things moving during the build.

Aymen – Tester: Aymen handled all the testing for the project. He built his own small prototype to manually run through the data structure operations and worked out a set of test cases with expected outcomes, which he committed to the repo. That early work fed directly into the MSTest unit and integration tests that were written for the final system. He then put together the full system test log covering regular user actions, admin functionality, and system-wide behaviour, ran every test case, wrote up the pass/fail results, and flagged anything that needed fixing to the dev team. He also put together a mobile UI prototype early on that helped shape the design direction.

Design

Justification of Data Structures

AVL Tree (Title Search) Chosen for guaranteed **O(log n)** search, insert, and delete operations. Ensures balanced performance even with large datasets.

Hash Table (Video ID Lookup) Provides **O(1)** average lookup time for video IDs. Essential for fast rental/return operations.

SQL Database (Persistence) Stores video metadata, rental records, and user accounts. Ensures data persistence across sessions.

WinForms UI Chosen for simplicity, speed, and coursework compliance. UI directly based on team Figma prototypes.

Algorithm Analysis (Pseudocode)

AVL Tree Insert

```
INSERT(node, key):
  if node is null:
    return new Node(key)
  if key < node.key:
    node.left = INSERT(node.left, key)
  else if  key > node.key:
    node.right = INSERT(node.right, key)
  else:
    return node
  UPDATE_HEIGHT(node)
  balance = GET_BALANCE(node)
  if balance > 1 and key < node.left.key:
   return RIGHT_ROTATE(node)
  if balance < -1 and key > node.right.key:
    return LEFT_ROTATE(node)
  if balance > 1 and key > node.left.key:
    node.left = LEFT_ROTATE(node.left)
    return RIGHT_ROTATE(node)
  if balance < -1 and key < node.right.key:
    node.right = RIGHT_ROTATE(node.right)
    return LEFT_ROTATE(node)
  return node
```

Hash Table Insert

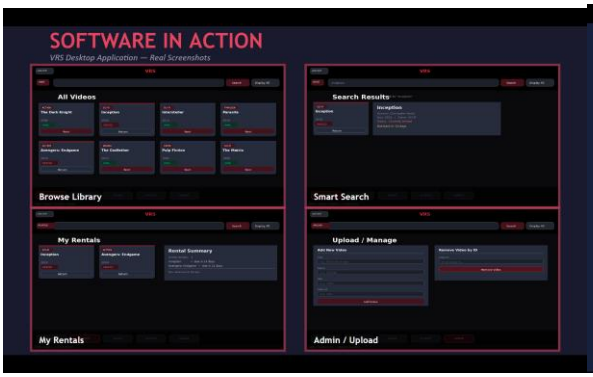
```
INSERT(key, value):
  index = HASH(key)
    while table[index] is occupied:
      index = (index + 1) mod TABLE_SIZE
        table[index] = value
```

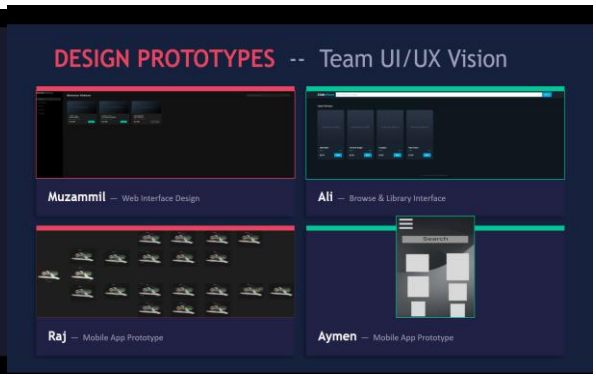
Rental Logic

```
RENT(videoID):
video = HashTable.Lookup(videoID)
  if video.status == "RENTED":
    return "Already rented"
    video.status = "RENTED"
      video.dueDate = TODAY + 28 days
```

Design Ideas

- Group-

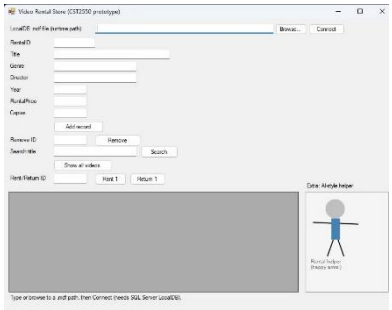




Figma prototype link - https://www.figma.com/design/JwCdMFKkx057eP2BpHmuhI/Untitled?node-id=475-1009&t=wjKxZWJZKvWXCrNm-1

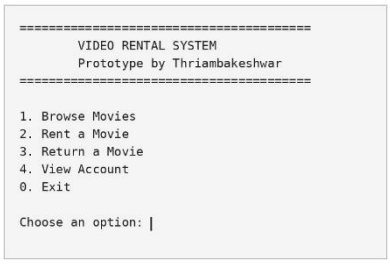
Aymen's WinForms Prototype

WinForms app connecting to a local .mdf DB. Full CRUD, rent/return, and an AI helper panel.



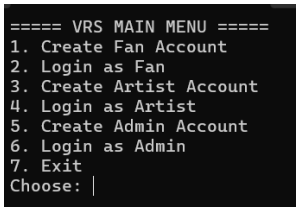
Thriambakeshwar's Console Prototype

Terminal-based VRS with menu-driven browse, rent, return, and account management.

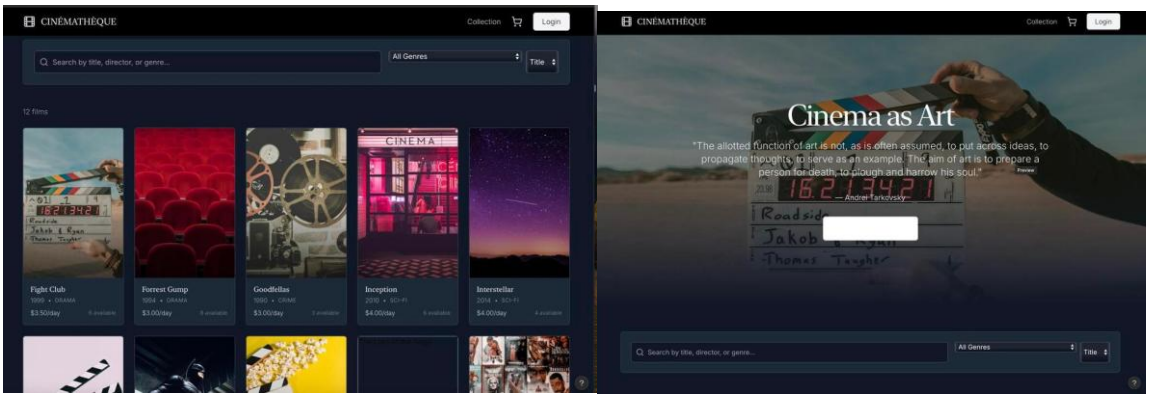


Raj's Console Prototype

Command-line VRS prototype. Fan, artist, and admin account flows all built in the terminal.



Muzammil's Web API Prototype



Testing

Testing Approach

- Unit Testing (AVL, Hash Table, Rental Logic)
- Integration Testing (SQL, CRUD, Login)
- User Acceptance Testing (Rent, Return, Search, UI Flow)

Test Case Table *Section 1 – Regular User*

Tests from the perspective of a standard logged-in user.

ID	*Test*	*Steps*	*Expected Result*	*Pass/Fail*	*Notes*
Login					
U-01	*Valid login*	*1. Go to login page. 2. Enter valid email and password. 3. Click Login.*	*User is logged in and taken to their home screen.*	*Pass*	*User is able to login, when all credentials are correct*

ID	Test	Steps	Expected Result	Pass/Fail	Notes
U-02	Wrong password	1. Enter valid email. 2. Enter incorrect password. 3. Click Login.	Login fails. Error message shown. User stays on login page.	Pass	It says a new status update that says invalid login.
U-03	Wrong username / email	1. Enter an email that does not exist. 2. Enter any password. 3. Click Login.	Login fails. Error message shown.	Pass	Same as the password, shows new status update that says invalid login.
U-04	Account lockout	1. Enter wrong password 5 times in a row for the same account.	Account is locked. User cannot attempt further logins for a set period.	Fail	User can indefinably attempt failure
U-05	Lockout message shown	1. Trigger lockout as above. 2. Read on-screen message.	Clear message shown explaining the account is locked.	N/A	User cannot be locked out
U-06	Logout	1. Log in successfully. 2. Click Logout.	Session ends. User is returned to login page and cannot navigate back without logging in again.	Pass	User can successfully logout
Account & Registration					
U-07	Duplicate email blocked	1. Register with an email already in the system.	Registration fails. Error message about duplicate email shown.	Fail	Just fails account creation but doesn't display error.
Favourites					
U-09	Favourite a video	1. Find a video. 2. Click the favourite / bookmark icon.	Video is added to the user's favourites list.	Pass	User can favourite videos
U-10	View favourites list	1. Navigate to Favourites section.	All favourited videos are listed.	Pass	User can visit favourite list

ID	Test	Steps	Expected Result	Pass/Fail	Notes
U-11	Remove a favourite	1. Open favourites. 2. Click unfavourite on a video.	Video is removed from the favourites list.	Pass	User can unfavourite a video
Search					
U-12	Search by keyword	1. Enter a keyword in the search bar. 2. Submit.	Matching video titles are returned.	Fail	User must search the complete title, to find it via search bar.
U-13	No search results	1. Search for a term that does not exist.	Empty result shown. Friendly message displayed.	Pass	Nothing displays
U-14	Search is not case sensitive	1. Search the same title in upper and lower case.	Same results returned both times.	fail	Search is case sensitive
Renting					
U-15	Rent a video	1. Find an available video. 2. Select rent. 3. Confirm.	Rental is created. Video appears in the user's rented list.	Pass	User can rent a video
U-16	Rented video shows expiry date	1. Rent a video. 2. Open rented videos list.	Expiry / due date is clearly shown next to the rental.	Pass	Shows count down until it expirers
U-19	Return a rented video	1. Open a current rental. 2. Select return. 3. Confirm.	Rental is marked as returned. Video is removed from active rentals.	Pass	User can return a rented video
U-20	Returned video no longer in active rentals	1. Return a video. 2. Check rented videos list.	Returned video no longer appears in active rentals.	Pass	Returned videos leave rental section
U-21	View rental history	1. Navigate to rental history.	Past rentals including returned ones are listed with dates.	Fail	No rental history records

Section 2 – Admin User*Tests from the perspective of an admin/staff account.*

ID	Test	Steps	Expected Result	Pass/Fail	Notes
Login					
A-01	Valid admin login	1. Enter valid admin username and password. 2. Click Login.	Admin is logged in and taken to the admin dashboard.	Pass	Allows login
A-02	Wrong password	1. Enter valid admin username. 2. Enter wrong password. 3. Click Login.	Login fails. Error message shown.	Pass	Fails and shows error
A-03	Wrong username	1. Enter an unknown username. 2. Enter any password. 3. Click Login.	Login fails. Error message shown.	Pass	Fails and shows error
A-04	Account lockout	1. Enter wrong credentials 5 times for the same admin account.	Account locked. Further attempts blocked for a set period.	Fail	Nothing happens
A-05	Logout	1. Log in as admin. 2. Click Logout.	Session ends. Admin returned to login page. Cannot navigate back without logging in again.	Pass	Meets requirements
Add Video Title					
A-06	Add a complete video title	1. Open Add Video. 2. Fill in ID, title, genre, release year. 3. Save.	Video title saved and visible in the catalog.	Pass	Allows video title to be saved and displayed
A-07	ID is assigned / accepted	1. Add a video with a specific ID. 2. Save. 3. Check record.	ID is stored and associated with the title.	Pass	Stores and saves ID
A-08	Title field accepted	1. Enter a title. 2. Save.	Title stored correctly.	Pass	Stores title well
A-09	Genre field accepted	1. Enter or select a genre. 2. Save.	Genre stored correctly.	Pass	Stores genre well

ID	Test	Steps	Expected Result	Pass/Fail	Notes
A-10	Release year accepted	1. Enter a release year. 2. Save.	Release year stored correctly.	Pass	Release year stored correctly
A-11	Skip ID – is it blocked or auto-assigned?	1. Add video without entering an ID. 2. Save.	System either blocks save with an error OR auto-assigns a unique ID.	Pass	System doesn't save without ID
A-12	Skip title – blocked	1. Leave title blank. 2. Save.	Save is blocked. Required field error shown.	Pass	System doesn't save
A-13	Skip genre – allowed or blocked?	1. Leave genre blank. 2. Save.	System either saves without genre or shows an error. Behaviour noted.	Fail	N/A
A-14	Skip release year – allowed or blocked?	1. Leave release year blank. 2. Save.	System either saves without year or shows an error. Behaviour noted.	Fail	N/A
A-15	Duplicate ID blocked	1. Add a video using an ID that already exists.	Save blocked. Duplicate ID error shown.	Pass	Tells user ID already exists.
Remove & Search Videos					
A-16	Remove video by ID	1. Enter a valid video ID in the remove field. 2. Confirm delete.	Video is removed from the catalog.	Pass	Removes video associated with entered ID
A-17	Remove video – ID not found	1. Enter an ID that does not exist. 2. Attempt delete.	Error shown. Nothing is deleted.	Pass	Status update: Not found
A-18	Search video by ID	1. Enter a valid video ID in search. 2. Submit.	Correct video title is returned.	Failed	Nothing is returned
A-19	Search by ID – not found	1. Enter an ID that does not exist. 2. Submit.	No results shown.	N/A	N/A

ID	Test	Steps	Expected Result	Pass/Fail	Notes
A-20	Cannot remove video with active rentals	1. Try to remove a video that is currently rented.	Deletion blocked. Error message shown.	Fail	Video is deleted
Admin User Management					

Section 3 – General System

System-wide tests covering search, UI, sessions, and stability.

ID	Test	Steps	Expected Result	Pass/Fail	Notes
User Interface					
S-07	Pages load without errors	1. Navigate to login, home, catalog, and account pages.	All pages load fully with no visible errors or broken elements.	Pass	No page load failures
S-08	Buttons are clickable and respond	1. Click all main buttons on each page.	Each button triggers the expected action with no lag or failure.	Pass	Buttons are functional
S-09	Navigation links work	1. Click all navigation menu items.	Each link navigates to the correct page.	Pass	Navigation is responsive and works
S-10	Error messages are readable	1. Trigger a validation error. 2. Read the message.	Message is plain English with no technical codes or stack traces.	Pass	Well sized text and fonts
S-11	Forms clear after submission	1. Submit a form successfully. 2. Check the fields.	Form fields are cleared or reset after a successful submission.	Pass	Once submitted forms clear out.
S-13	UI works on different screen sizes	1. View the app on a small screen and a large screen.	Layout adjusts correctly. No content is cut off or overlapping.	Pass	UI adjusts
Session & Security					

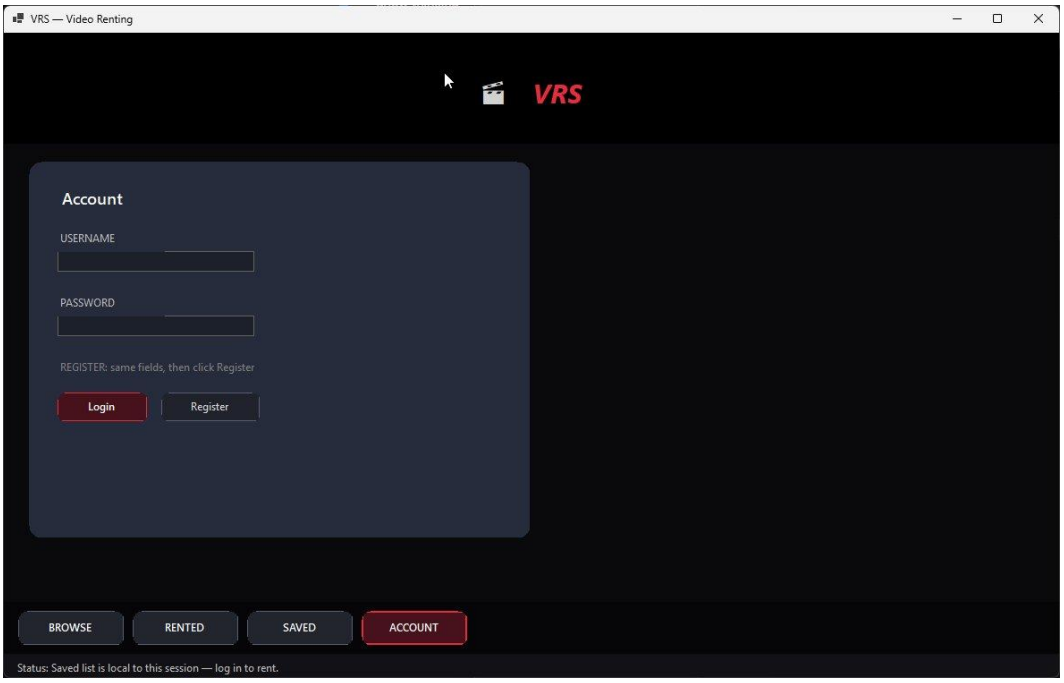
ID	Test	Steps	Expected Result	Pass/Fail	Notes
S-16	*Password field is masked*	*1. Go to login page. 2. Type in the password field.*	*Password characters are hidden (shown as dots or asterisks).*	*Pass*	*Shows dots in place of letters*
Data & Stability					
S-17	*App does not crash on invalid input*	*1. Enter special characters and very long strings into text fields. 2. Submit.*	*App handles input gracefully. No crash or unhandled error.*	*Pass*	*No crash*
S-18	*Confirmation shown after key actions*	*1. Rent a video, return a video, update account details.*	*A confirmation message or screen is shown after each action.*	*Pass*	*Shows confirmation*
S-19	*Data persists after page refresh*	*1. Favourite a video. 2. Refresh the page.*	*Favourite is still saved after refresh.*	*Pass*	*Data persists*

Software Screenshots

This section documents both generations of the VRS interface. The first set of screenshots shows the original WinForms desktop prototype. The second set shows the final React and ASP.NET Core Web API implementation, which replaced WinForms after the team identified that a browser-based, multi-user platform was a better fit for the system's goals. Screenshots are provided for each user role: Customer, Publisher, and Admin, as well as the AI Chatbot assistant.

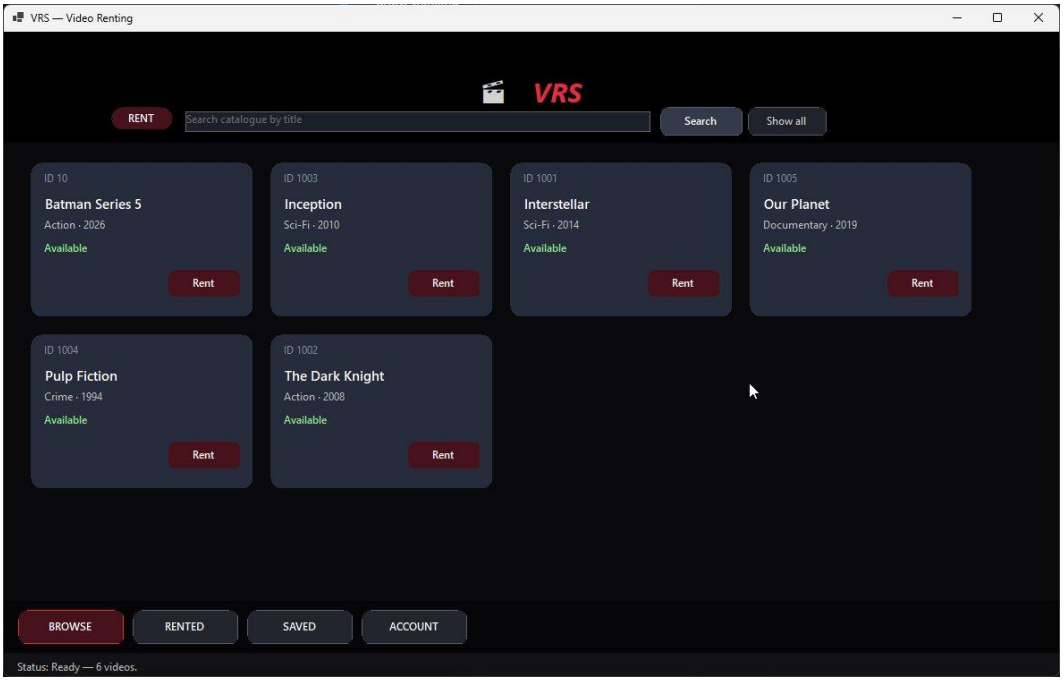
Login Screen

The login screen in the WinForms prototype. Users enter a username and password to sign in. There's also a separate admin login. If credentials are wrong, it shows an error without saying whether it was the username or password that was off.



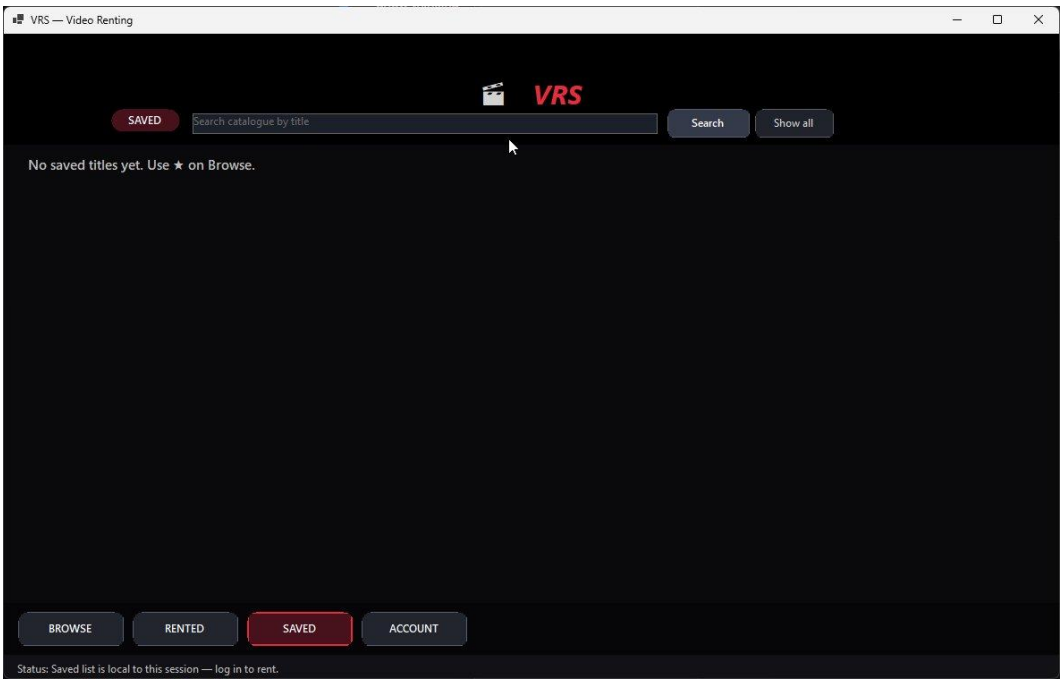
Browse Videos

The browse screen in WinForms showing the full catalog pulled from the database through the AVL tree. Users can filter by keyword, genre, or price. Each entry shows the title, genre, publisher, price, and whether it's available. Unpublished titles don't show up here.



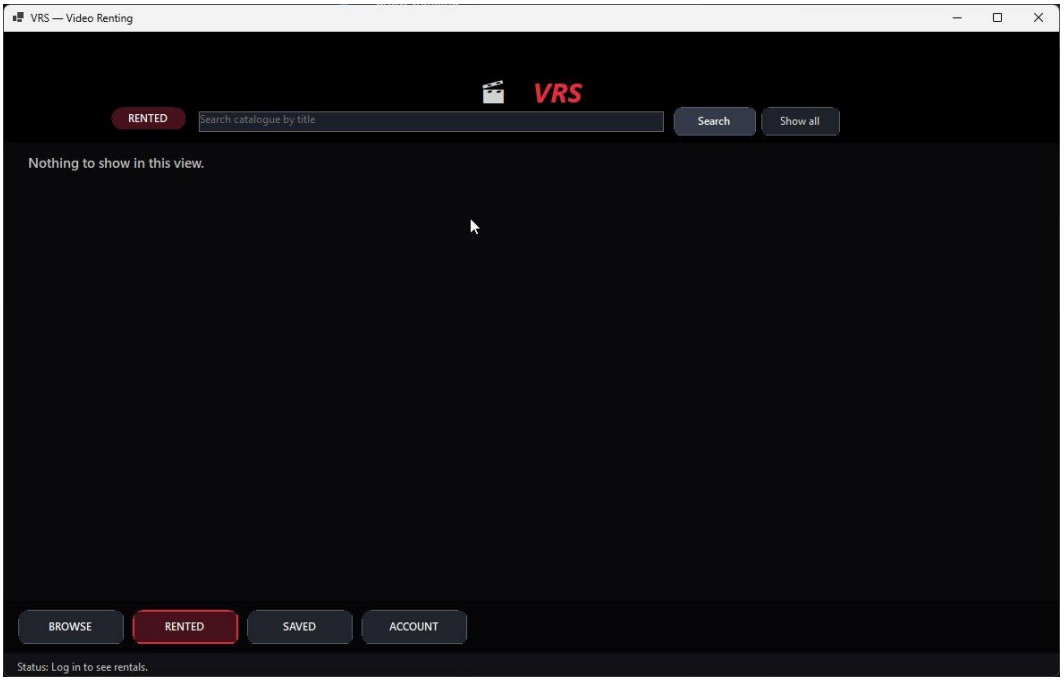
Search Results

Search results in the WinForms build, coming back through the AVL tree. Results update as you type or change the filters. The hash table is used on the backend to resolve each result to its full video record.



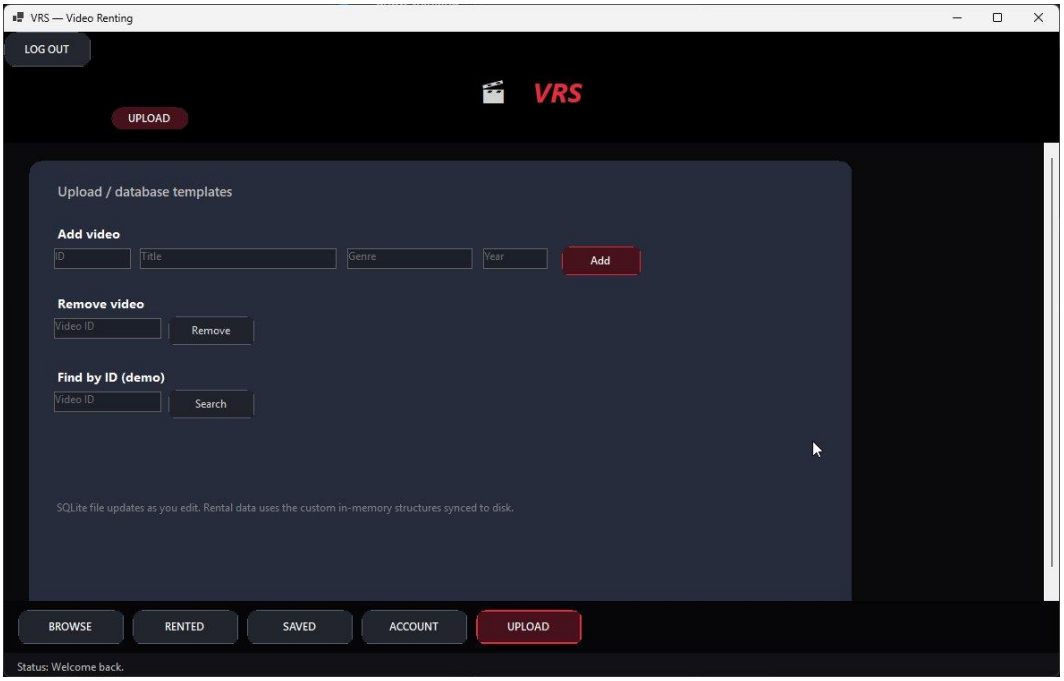
Rent / Return Workflow

The rent and return workflow allows authenticated customers to rent an available video by clicking Rent on its listing. The system records the rental in the SQL database and marks the video as unavailable. On return, the customer selects the video from their active rentals list and confirms the return, restoring the video's availability. Admins can also perform rent and return operations on behalf of any user through the admin panel.



Admin Upload Panel

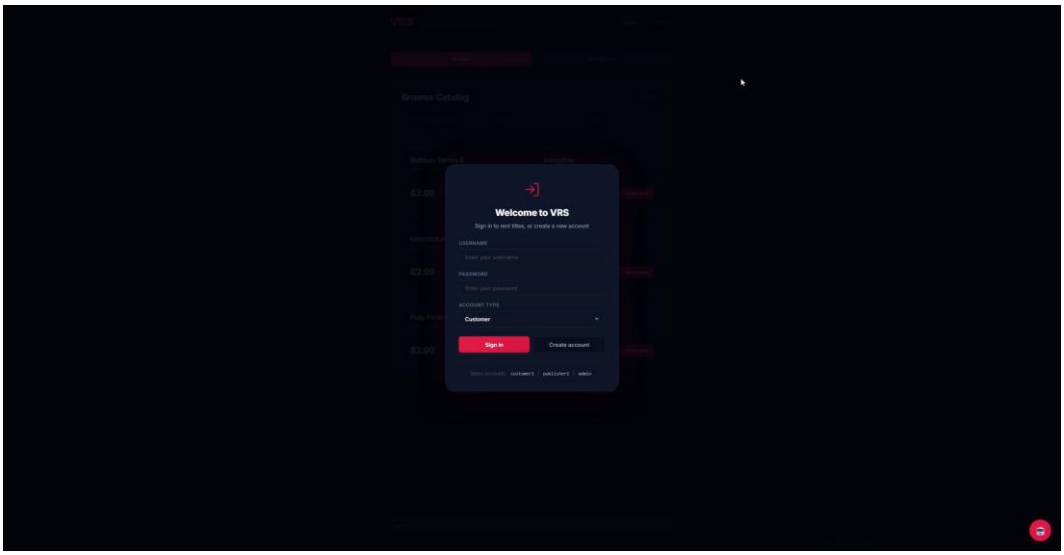
The WinForms admin panel. Only accessible through the admin login, regular accounts get rejected. From here you can manage users, add/edit/delete videos, toggle publish status, and view all rentals across the system. Everything writes straight to the database.



Web API Frontend Screenshots

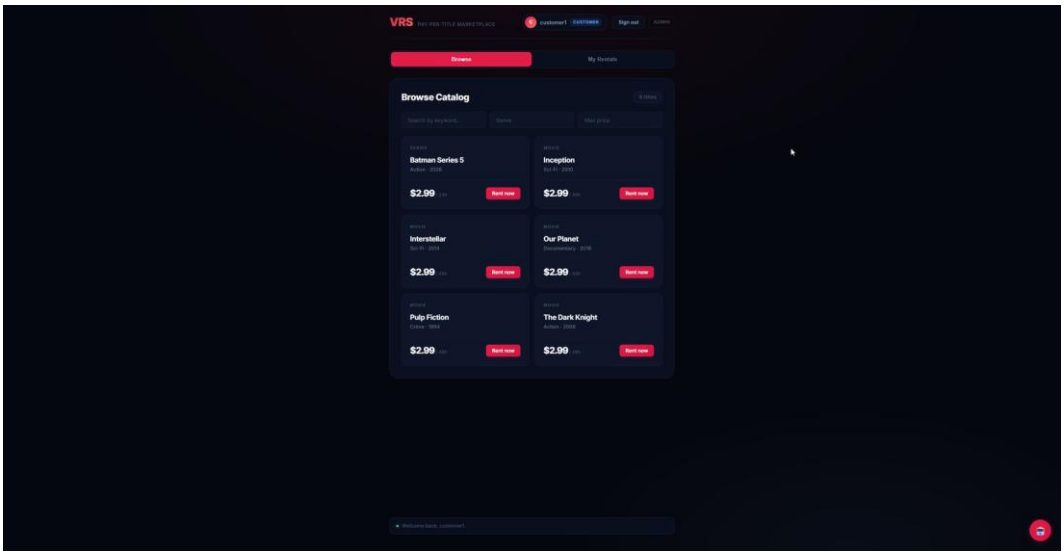
Login Screen (Web API – All Roles)

The login modal on the web version. It pops up over the catalog so you can see what you're signing into. You pick your account type from the dropdown. Demo accounts are shown at the bottom for quick testing.



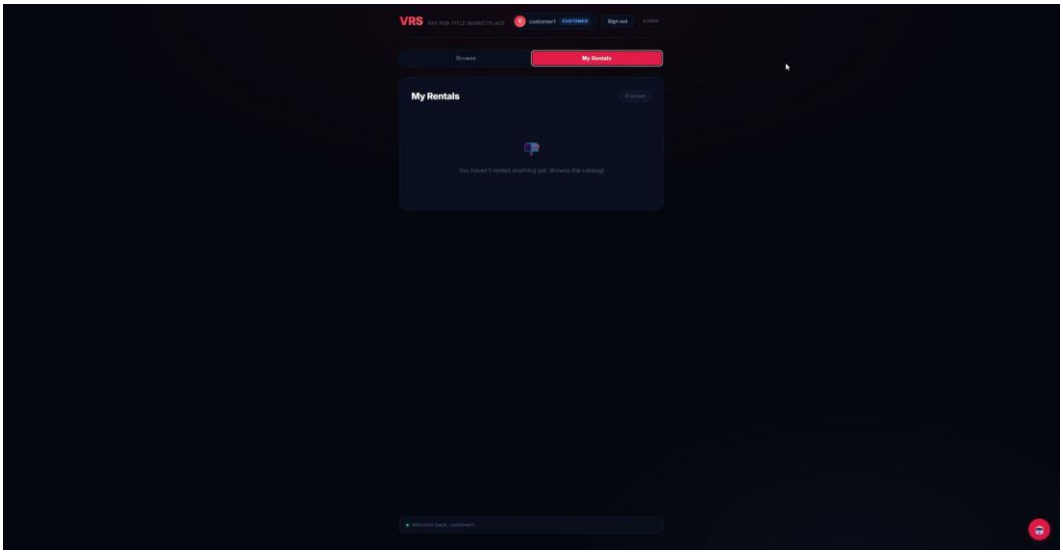
Browse Catalog – Logged in as Customer (customer1)

Browse catalog view logged in as customer1. Customer accounts only see Browse and My Rentals in the nav. Six titles are showing here with price, duration, and a Rent now button on each.



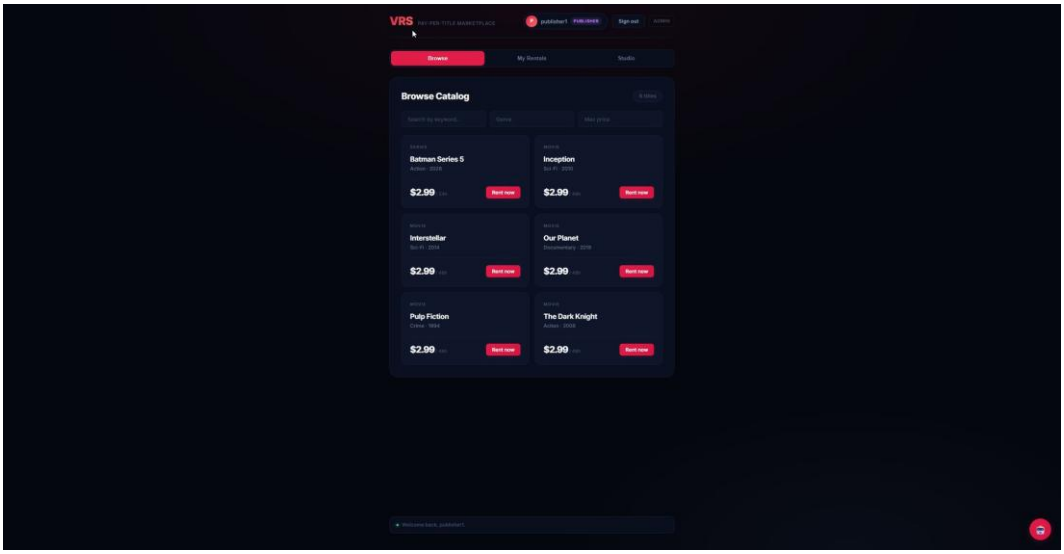
My Rentals – Logged in as Customer (customer1)

My Rentals for customer1, nothing rented yet so it shows an empty state. The count updates live when you rent or return something.



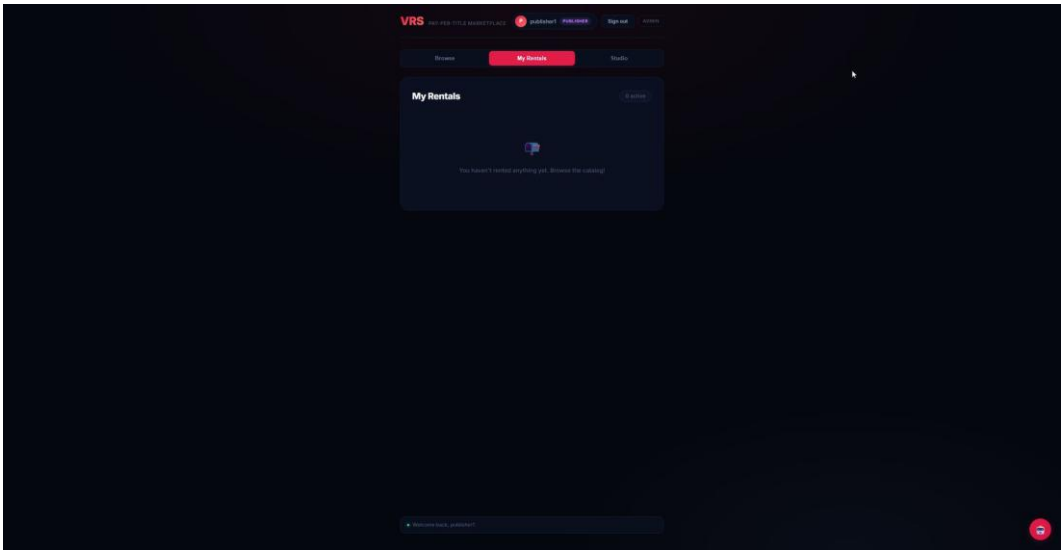
Browse Catalog – Logged in as Publisher (publisher1)

Browse catalog logged in as publisher1. Publishers get a third tab, Studio, in the nav alongside Browse and My Rentals.



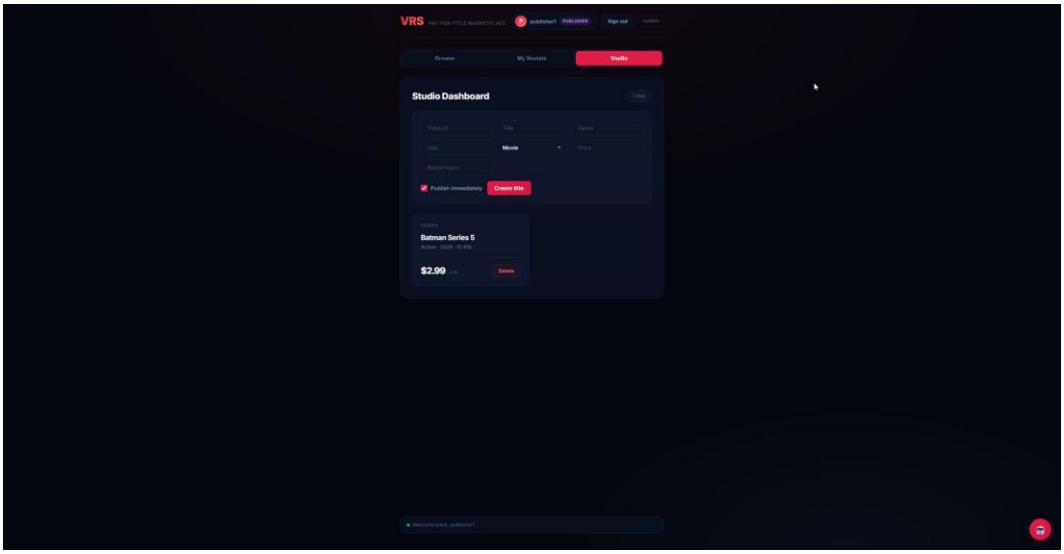
My Rentals – Logged in as Publisher (publisher1)

My Rentals for publisher1. Publishers can rent titles from other studios the same way a customer would – it's completely separate from their publishing side.



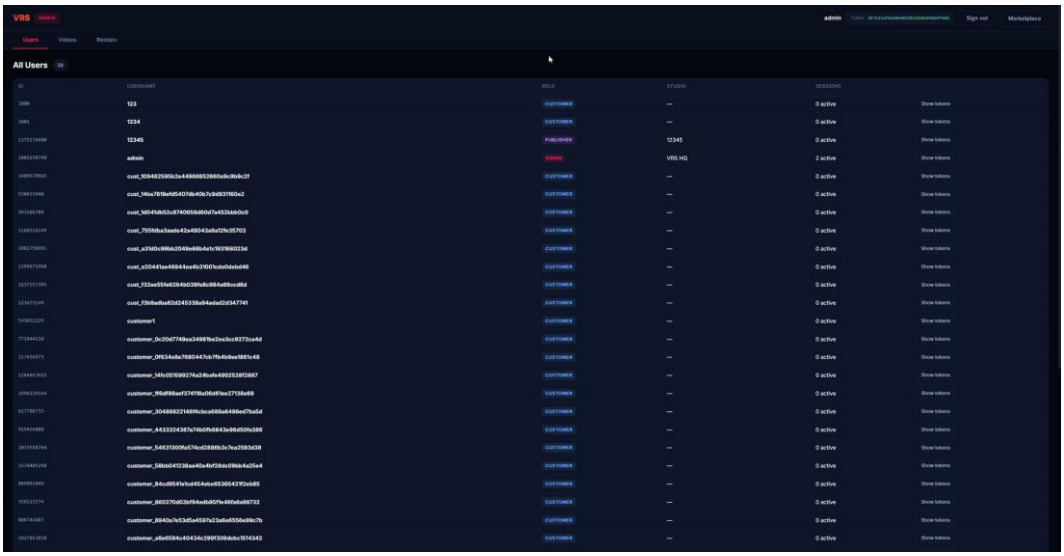
Studio Dashboard – Logged in as Publisher (publisher1)

The Studio Dashboard for a publisher. Fill in the details and hit Create title. The Publish immediately checkbox decides whether it goes live straight away or sits as a draft. Existing titles show below with a Delete option.



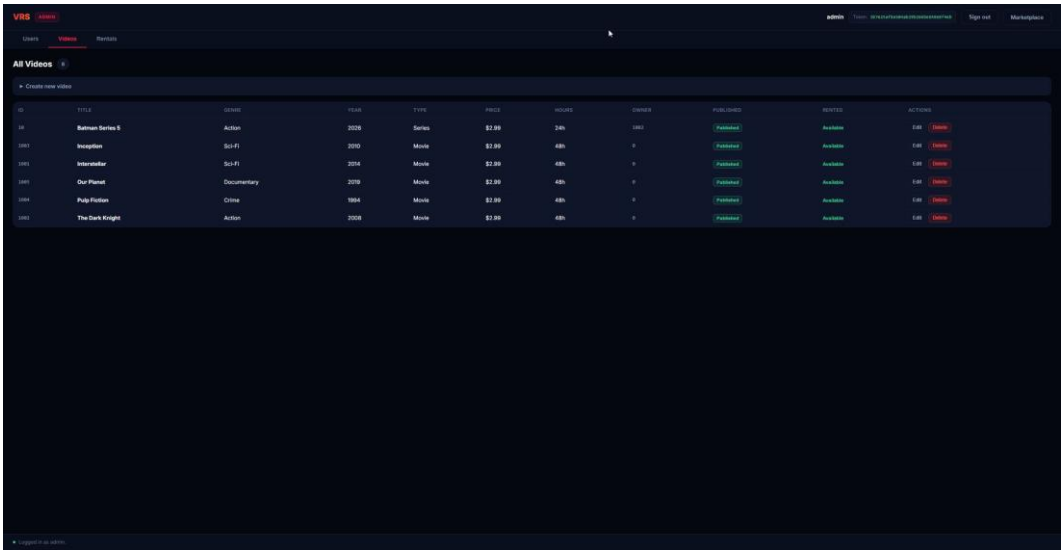
Admin Panel – Users Tab (Logged in as Admin)

Admin panel – Users tab. Shows all 39 users with their ID, username, role, studio (if they have one), and how many active sessions they have. The Show tokens button lets admins see live session tokens.



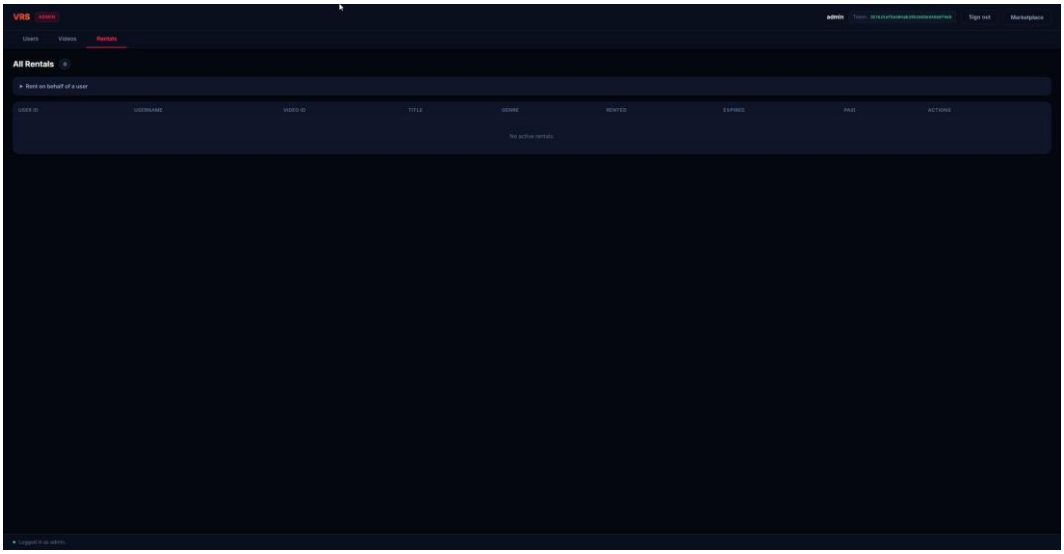
Admin Panel – Videos Tab (Logged in as Admin)

Admin panel – Videos tab. All 6 titles listed with their full details. Admins can create, edit inline, or delete any title from here.



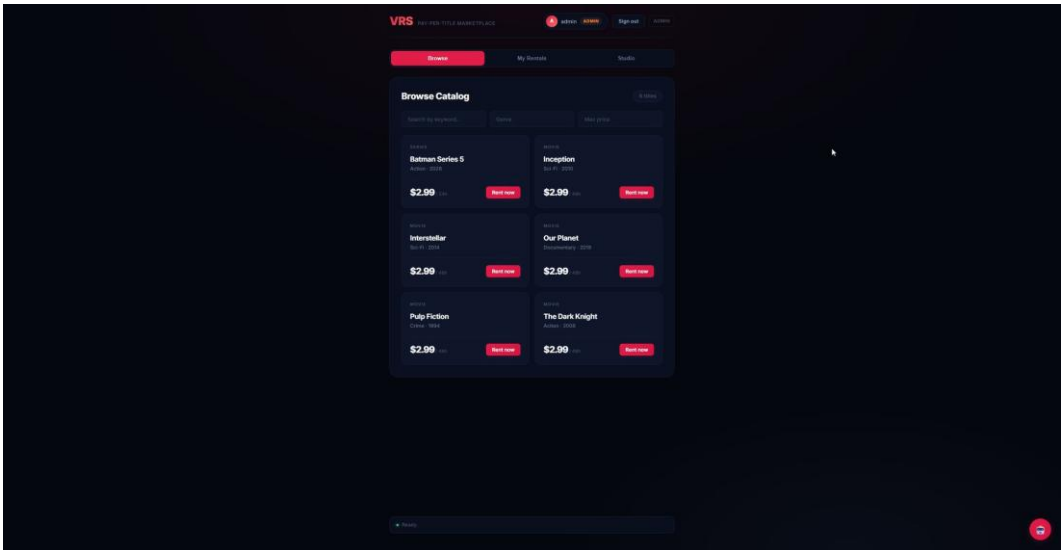
Admin Panel – Rentals Tab (Logged in as Admin)

Admin panel, Rentals tab. Shows all active rentals across every user. There's also a Rent on behalf of a user option so admins can manually create rentals for customers.



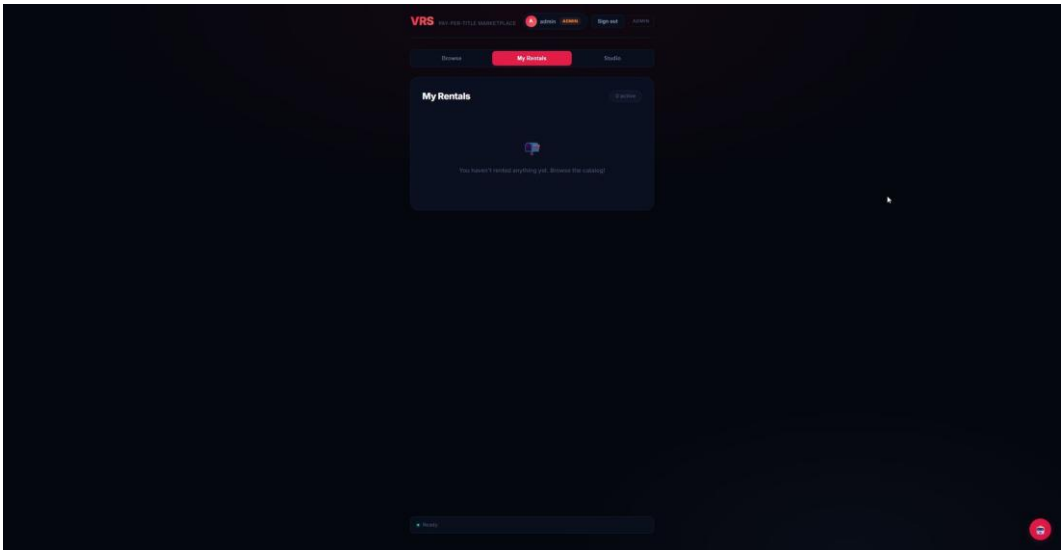
Browse Catalog – Logged in as Admin

Browse catalog logged in as the admin account. Admins have access to Browse, My Rentals, and Studio all at once.



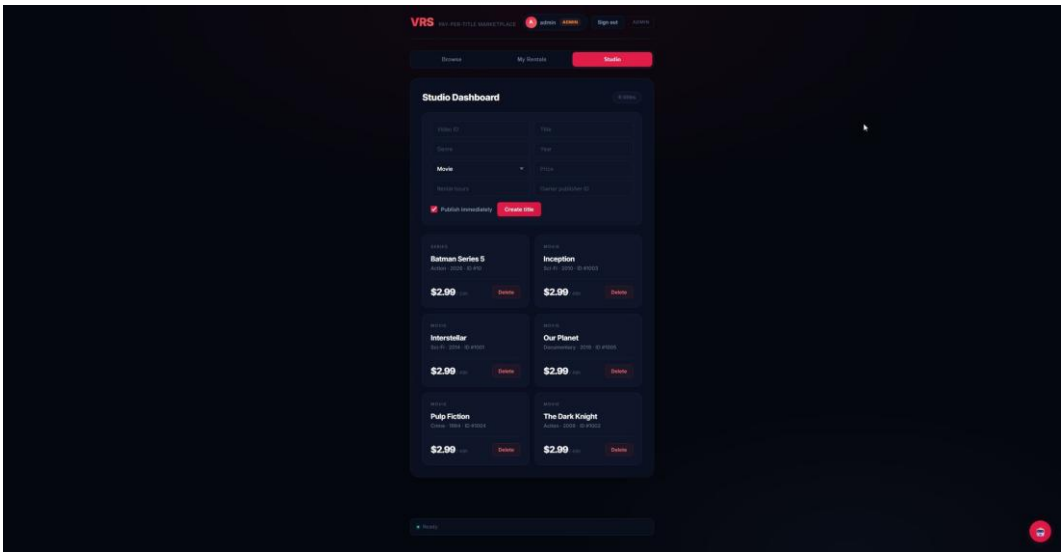
My Rentals – Logged in as Admin

My Rentals for the admin account – this is just the admin's own rental history, separate from the system-wide rentals view in the admin panel.



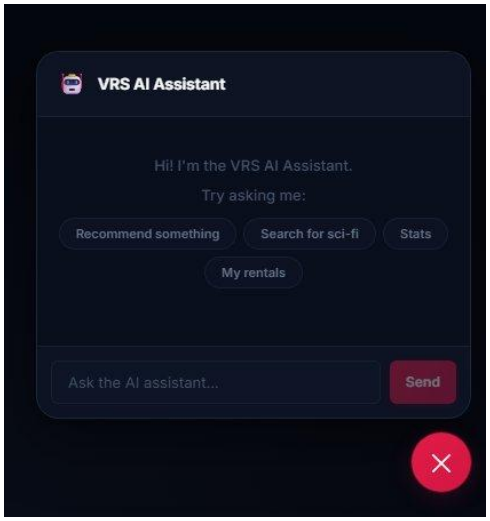
Studio Dashboard – Logged in as Admin

Studio Dashboard for the admin. Shows all 6 titles on the platform. The create form has an Owner publisher ID field so admins can assign new titles to any publisher, not just themselves.



AI Chatbot – VRS AI Assistant

The AI chatbot, a floating button available on every page. You can ask it things in plain English like "recommend something" or "what sci-fi do you have". It knows what's in the catalog and who you're logged in as. This was the optional AI agent extra marks piece.



Conclusion

Summary of Work Done

The team built and tested a working VRS using C# .NET. The project started with a WinForms prototype but the team switched to a React/TypeScript frontend and ASP.NET Core Web API once it became clear that a desktop app wasn't going to work for a multi-user rental platform. The custom data structures, AVL tree, hash table, BST, and hash maps, were all written from scratch with no STL collections used. SQLite handles the database side and the system persists all user, video, and rental data between sessions. Validation is in place throughout and no file names are hardcoded. The AI chatbot was also built in as the optional extra marks feature. All team members committed to the repo regularly and MSTest was used for the unit and integration tests.

Limitations and Critical Reflection

The main thing that didn't fully work was the lockout on failed logins, the system validates credentials fine and shows error messages, but it doesn't block you after too many wrong attempts. That was left out because of time at the end of the project and would be the first thing to fix. Session tokens are also stored in memory rather than the database, which means a server restart kills all active sessions. Not ideal for a real deployment. The WinForms and web versions were being built at the same time which caused some inconsistencies in how the business logic worked between the two. That wouldn't happen if there was a single shared core library from the start. Merge conflicts were also a recurring issue because the team wasn't using proper feature branches.

How We Would Change Our Approach in Future

Next time the team would use proper feature branches from day one instead of everyone pushing to main. Pull requests before merging would have stopped most of the conflicts. It would also help to agree on the core data structure interfaces before anyone starts writing code, so there's no duplicated logic between different parts of the system. Testing and documentation kept getting pushed to the end of each sprint when time was tight, building those into the sprint from the start would mean they get done properly. It would also be worth having one person specifically responsible for integration testing rather than leaving it as something everyone sort of owns but nobody really does.

References

Adelson-Velsky, G. and Landis, E. (1962) 'An algorithm for the organisation of information', Soviet Mathematics Doklady, 3, pp. 1259–1263.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.

Knuth, D.E. (1998) The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd edn. Reading, MA: Addison-Wesley.

Microsoft (2024) ASP.NET Core documentation. Available at: https://learn.microsoft.com/en-us/aspnet/core/ (Accessed: 9 April 2026).

Microsoft (2024) C# documentation. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/ (Accessed: 9 April 2026).

SQLite (2024) SQLite documentation. Available at: https://www.sqlite.org/docs.html (Accessed: 9 April 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 9 April 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) Experimentation in Software Engineering. Berlin: Springer.